

Arduino de cero a sólido

Manual pedagógico para entender placas, sensores, protoboard, resistencias y electrónica — con todo lo necesario para construir y enseñar el prototipo GeoGreen.

★ Tipos de placas

★ Sensores

★ Protoboard

★ Resistencias

★ Programación

★ IoT/ESP32 (intro)



Guía de estudio · escrita para Diego Obando · Escuela de Programación y Análisis de Sistemas.
Hardware abierto · todos los ejemplos son simulables en Wokwi sin placa física.

Índice

1 ¿Qué es Arduino?

Microcontrolador, hardware abierto y ecosistema

2 Tipos de placas Arduino

UNO, Nano, Mega, R4, ESP32... cuál elegir

3 Anatomía de la placa (UNO)

El chip, las memorias y todos los pines

4 Electricidad esencial

Voltaje, corriente, resistencia y Ley de Ohm

5 La protoboard a fondo

Cómo conecta por dentro y cómo cablear

6 Resistencias

Para qué sirven, cálculo y código de colores

7 Componentes básicos

LED, botón, potenciómetro, transistor, relé, buzzer

8 Sensores

El catálogo: qué miden y cómo se conectan

9 Actuadores

Motores, servos, relés y pantallas

10 Protocolos de comunicación

Serial, I2C y SPI

11 Programación de Arduino

setup/loop y ejemplos comentados

12 IoT y ESP32 (introducción)

WiFi, 3,3V y dashboards

13 Caso práctico: GeoGreen

El circuito completo integrado

14 Seguridad y buenas prácticas

Para no quemar nada

15 Glosario y recursos

Términos clave y dónde seguir

CÓMO LEER ESTA GUÍA

Cada capítulo arranca con la idea grande, usa **analogías** para lo abstracto, y cierra con **mini-ejercicios**. Los recuadros verdes (**GeoGreen**) conectan la teoría con nuestro prototipo. No necesitas placa física: todo se puede probar en wokwi.com.

¿Qué es Arduino?

Arduino es una **plataforma de electrónica de hardware abierto** pensada para que cualquiera pueda crear objetos que sienten y controlan el mundo físico, sin ser ingeniero electrónico.

Cuando alguien dice "Arduino" puede referirse a tres cosas que trabajan juntas:

- **La placa** (hardware): una tarjeta con un **microcontrolador** y conectores (pines).
- **El IDE** (software): el programa donde escribes el código y lo "subes" a la placa por USB.
- **El lenguaje**: una versión simplificada de **C/C++**, con funciones fáciles como `digitalWrite()`.

Microcontrolador vs. microprocesador

Tu PC tiene un **microprocesador**: muy potente, pero necesita memoria, disco, sistema operativo... Un **microcontrolador** (el corazón del Arduino) es un "computador completo en un solo chip": trae CPU + memoria + entradas/salidas, es barato y consume poquísimo. No corre Windows; corre **un único programa** que tú le grabas, una y otra vez, para siempre.

ANALOGÍA

El microprocesador es un **cerebro de oficinista** (hace mil cosas, necesita escritorio, café y archivos). El microcontrolador es un **cerebro de reflejo**: hace una tarea concreta, rapidísimo, sin distraerse y casi sin gastar energía. Para "leer un sensor y prender un LED", el reflejo sobra.

¿Por qué "hardware abierto"?

Los planos de las placas son públicos: cualquiera puede fabricarlas o mejorarlas. Por eso existen miles de placas "compatibles con Arduino" más baratas, y una enorme comunidad con tutoriales, librerías y ejemplos gratis. Para enseñar, esto es oro: hay material para todo.

GEOGREEN

Nuestro prototipo usa un Arduino para **leer un sensor de distancia**, calcular el **% de llenado** de un contenedor y encender un **semáforo** (verde/amarillo/rojo) + un **buzzer** de alerta. Todo lo de esta guía existe para que entiendas ese circuito por dentro.

Ejercicios

1. Explica con tus palabras la diferencia entre microcontrolador y microprocesador.
2. Nombra las tres cosas que componen "Arduino" (hardware, software, lenguaje).
3. ¿Por qué crees que para GeoGreen conviene un microcontrolador y no un PC?

Tipos de placas Arduino

Todas comparten la misma idea (un microcontrolador con pines), pero cambian en **tamaño, potencia, cantidad de pines y conectividad**. Elegir bien es la mitad del proyecto.

Arduino agrupa sus placas en tres familias:

- **Entrada (Entry):** para aprender y prototipar — **UNO, Nano, Mega**.
- **IoT:** traen WiFi/Bluetooth — **UNO R4 WiFi, Nano ESP32, MKR, Nano 33 IoT, GIGA**.
- **IA / Pro:** potentes, para visión y machine learning — **Portenta, Nicla**.

Comparativa de las placas más usadas

Placa	Tamaño	Chip / Lógica	Pines (dig/ana)	Conectividad	Ideal para...
UNO R3	Grande (7×5,3 cm)	ATmega328P · 5V	14 / 6	USB (no red)	Aprender, talleres
UNO R4 Minima	Grande	Renesas RA4M1 32-bit · 5V	14 / 6	USB-C (no red)	UNO moderno, más memoria
UNO R4 WiFi	Grande	RA4M1 + ESP32-S3 · 5V	14 / 6	WiFi + BLE + matriz LED	IoT con formato UNO
Leonardo	Grande	ATmega32u4 · 5V	20 / 12	USB nativo (teclado/mouse)	Emular HID
Nano	Chica (4,5×1,8 cm)	ATmega328P · 5V	14 / 8	USB (no red)	Proyectos compactos
Nano ESP32	Chica	ESP32-S3 · 3,3V	~14 / 8	WiFi + BLE	IoT compacto
Mega 2560	Muy grande	ATmega2560 · 5V	54 / 16	USB (no red)	Muchos pines (robótica)
MKR / Nano 33 IoT	Chica	SAMD21 · 3,3V	~22 / 7	WiFi/LoRa/GSM	IoT a batería

DATO CLAVE: 5V VS 3,3V

La UNO/Nano clásicas trabajan a **5 voltios**; las placas IoT modernas (ESP32, SAMD) a **3,3 voltios**. Esto importa porque un sensor de 5V puede **dañar** un pin de 3,3V. Lo vemos en el Cap. 12.

GEOGREEN: CUÁL USAR

UNO R3 para el **taller** (grande, robusta, fácil de ver y conectar). **Nano** para el **módulo clip-on real** (chica, entra en la cajita bajo la tapa). Si más adelante queremos enviar datos por WiFi a un mapa, saltamos a **ESP32**.

Ejercicios

1. ¿Qué placa elegirías para un robot con 30 motores y sensores? ¿Por qué?
2. ¿Cuál es la diferencia práctica entre la UNO y la Nano?
3. ¿Qué ventaja da una placa IoT frente a la UNO clásica?

Anatomía de la placa (UNO)

Usaremos la UNO como referencia porque es la más didáctica. Conocer cada parte te permite leer cualquier tutorial sin perderte.

Las partes físicas

- **Microcontrolador (ATmega328P):** el cerebro. Guarda y ejecuta tu programa.
- **Conector USB:** sube el código desde el PC y alimenta la placa (5V).
- **Jack de alimentación:** permite alimentar con 7–12V (pila/fuente).
- **Regulador de voltaje:** convierte esa entrada a los 5V estables que usa todo.
- **Cristal (16 MHz):** el "metrónomo" que marca el ritmo del chip.
- **Botón Reset:** reinicia el programa desde cero.
- **LED integrado (pin 13 / LED_BUILTIN):** un LED en la placa para probar sin cablear nada.

Las memorias del chip

Memoria	Tamaño (UNO)	Qué guarda	Se borra al apagar?
Flash	32 KB	Tu programa	No
SRAM	2 KB	Variables mientras corre	Sí
EEPROM	1 KB	Datos que deben sobrevivir al apagado	No

El mapa de pines



Pinout simplificado de la UNO. ~ = pin con PWM. A4/A5 sirven también como bus I2C (SDA/SCL).

Los pines se agrupan por función:

- **ENERGÍA** **5V / 3V3** entregan voltaje; **GND** es el retorno (negativo); **VIN** alimenta la placa desde fuera.
- **DIGITAL** **D0–D13**: leen o escriben dos estados (5V/0V). Los marcados con ~ hacen PWM.
- **ANALÓGICO** **A0–A5**: leen voltajes variables (0–5V) → números 0 a 1023.
- **COMUNICACIÓN** **D0/D1** (Serial), **A4/A5** (I2C), **D10–D13** (SPI). Cap. 10.

CUIDADO

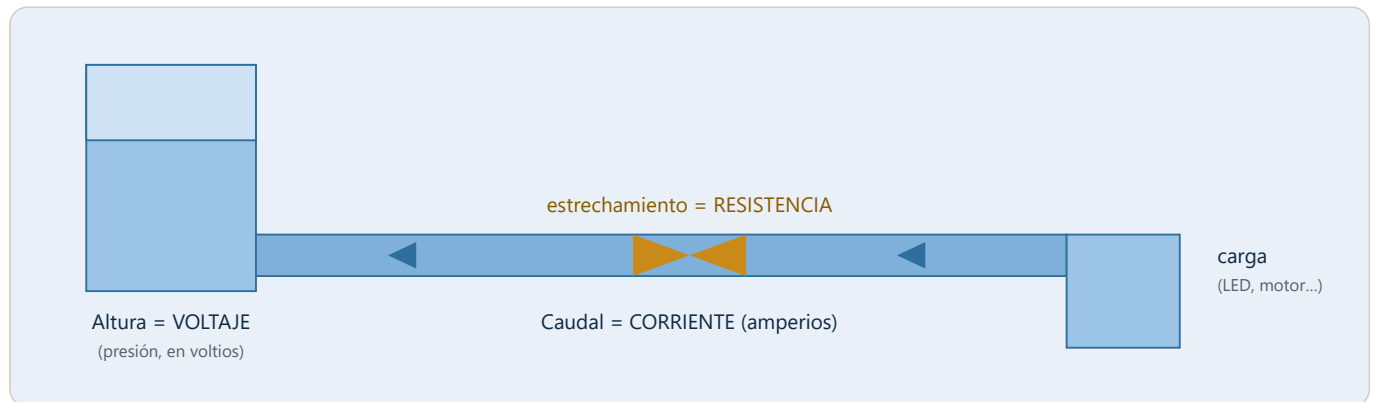
Cada pin entrega como máximo **~20–40 mA**. Sirve para LEDs y señales, **no** para motores o tiras LED (esos necesitan alimentación aparte + un transistor/relé). Forzarlo quema el pin o el chip.

Ejercicios

1. ¿Qué memoria guarda tu programa y cuál las variables temporales?
2. ¿Para qué sirve el símbolo  junto a algunos pines?
3. ¿En qué pines conectarías el bus I2C? ¿Y por qué no usar A0 para un LED de potencia?

Electricidad esencial

Tres conceptos explican el 90% de la electrónica de hobby: **voltaje**, **corriente** y **resistencia**. La mejor forma de entenderlos es con agua.



Voltaje = presión · Corriente = caudal · Resistencia = qué tan angosta es la cañería.

- **Voltaje (V), en voltios:** la "presión" que empuja la electricidad. La UNO trabaja a 5V.
- **Corriente (I), en amperios (A):** cuánta electricidad fluye. En electrónica suele medirse en miliamperios (mA).
- **Resistencia (R), en ohmios (Ω):** cuánto se opone un material al paso de la corriente.

La Ley de Ohm

La relación entre los tres es una sola fórmula, la herramienta más usada de toda la electrónica:

$$V = I \times R \rightarrow I = V / R \rightarrow R = V / I$$

Y la **potencia** (calor/energía por segundo), en vatios (W): **$P = V \times I$** . Por eso una resistencia que disipa mucho se calienta.

Digital vs. analógico

Digital = dos estados (encendido/apagado, 1/0, 5V/0V). **Analógico** = infinitos valores intermedios (como un dimmer). El Arduino lee analógico con `analogRead()` y lo convierte a un número de **0 a 1023**.

PWM: "fingir" lo analógico desde lo digital

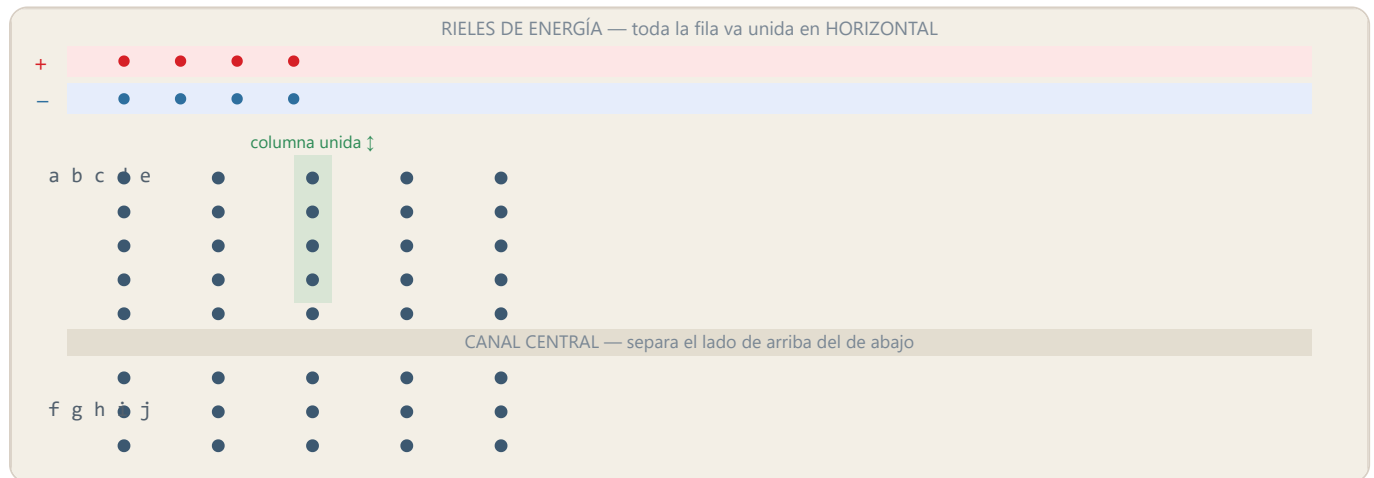
Los pines digitales solo dan 0V o 5V, pero con **PWM** (modulación por ancho de pulso) encienden y apagan miles de veces por segundo. Si está "encendido" el 30% del tiempo, el LED brilla al 30%. Se usa `analogWrite(pin, 0..255)`.

Ejercicios

1. Si tienes 5V y una resistencia de 1000 Ω , ¿cuánta corriente circula? (usa $I = V/R$)
2. Da un ejemplo cotidiano de señal digital y uno de señal analógica.
3. ¿Cómo logra un pin digital atenuar un LED si solo sabe prender/apagar?

La protoboard a fondo

La protoboard (breadboard) es una tabla para armar circuitos **sin soldar**: metes las patitas en los huecos y por dentro ya están conectadas de cierta forma. Entender ese "por dentro" es lo único que necesitas.



Por dentro: los rieles (+/-) van en horizontal; las columnas de 5 huecos van en vertical; el canal central separa ambos lados.

Las dos reglas de oro

1. **Las 5 columnas (a–e) están unidas en VERTICAL.** Dos patitas en la misma columna quedan conectadas (mismo "nodo"). Lo mismo abajo (f–j).
2. **El canal central separa arriba de abajo,** y los **rieles de los bordes (+ y –)** van en horizontal: sirven para repartir energía por toda la placa.

Cómo se cablea

Conectas **una sola vez** el 5V del Arduino al riel + y el GND al riel –. Desde ahí "tomas" energía donde quieras con **jumpers** (cables puente): macho-macho para protoboard, macho-hembra para ir a un sensor.

ERRORES TÍPICOS

- Poner las dos patitas de un componente en la **misma columna** → lo cortocircuitas.
- Olvidar el **GND común**: si el sensor y el Arduino no comparten tierra, nada funciona.
- Conectar al riel equivocado (+ en vez de –).

Ejercicios

1. Si metes un LED en la columna 4 (huecos a y c), ¿están conectados sus extremos? Explica.
2. ¿Para qué sirven los rieles rojo y azul de los bordes?
3. Dibuja cómo llevarías 5V y GND del Arduino a la protoboard.

Resistencias

Una resistencia **limita la corriente**. Es el componente que más asusta al principio y el más simple cuando lo entiendes: solo aplica la Ley de Ohm.

¿Por qué un LED necesita resistencia?

Un LED es "glotón": conectado directo a 5V deja pasar toda la corriente posible y **se quema**. La resistencia actúa como una llave que solo deja pasar lo seguro (~20 mA). Va **en serie** con el LED (uno detrás del otro).

El cálculo (de dónde sale el 220 Ω)

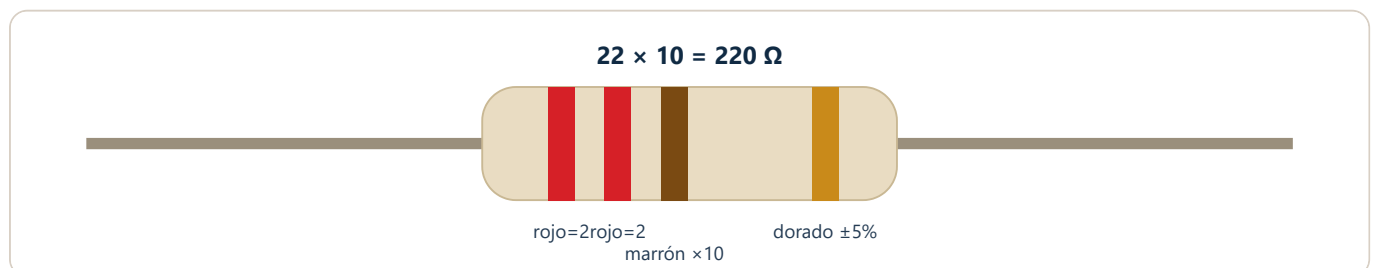
El LED "consume" parte del voltaje (un LED rojo ~2V) y quiere ~20 mA. La resistencia se come el resto del voltaje:

$$R = (V_{\text{fuente}} - V_{\text{LED}}) / I = (5 - 2) / 0,02 = \mathbf{150\ \Omega}$$

150 Ω sería el mínimo; se usa **220 Ω** (valor muy común) para ir holgados: el LED queda **brillante pero seguro**. Por eso GeoGreen usa 220 Ω en cada LED.

El código de colores

Las resistencias no traen el número impreso, sino **bandas de colores**. Para 4 bandas: 1ª y 2ª = dígitos, 3ª = multiplicador ($\times 10^n$), 4ª = tolerancia.

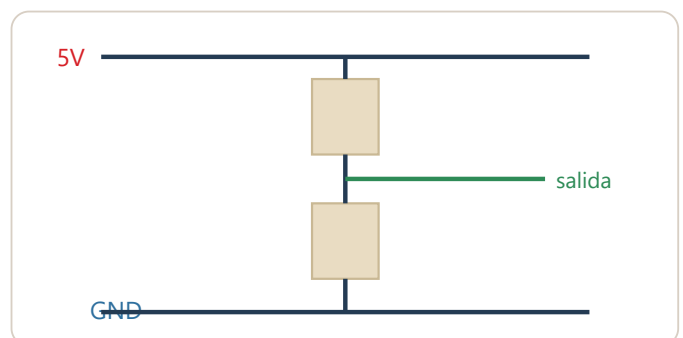


220 Ω = rojo · rojo · marrón · dorado. (rojo=2, rojo=2 → "22"; marrón= $\times 10$ → 220 Ω; dorado = tolerancia ±5%)

Color	Negro	Marrón	Rojo	Naranja	Amarillo	Verde	Azul	Violeta
Dígito	0	1	2	3	4	5	6	7

Serie, paralelo y divisor de voltaje

En serie (una tras otra) las resistencias se **suman**: 100 + 220 = 320 Ω. **En paralelo** el total **baja**. Y combinando dos en serie obtienes un **divisor de voltaje**: una forma de bajar un voltaje (por ejemplo, de 5V a 3,3V para proteger un ESP32 — Cap. 12).



Divisor: el punto medio entrega un voltaje menor.

RESISTENCIA PULL-UP / PULL-DOWN

Cuando un pin "queda al aire" lee valores basura. Una resistencia **pull-up** (a 5V) o **pull-down** (a GND) lo fija en un estado estable. Los botones siempre la necesitan; la UNO trae pull-ups internas (`INPUT_PULLUP`).

Ejercicios

1. Calcula la resistencia para un LED azul ($V_{led} \approx 3V$) a 20 mA con fuente de 5V.
2. ¿Qué valor tiene una resistencia con bandas marrón-negro-rojo?
3. ¿Por qué un botón necesita resistencia pull-up o pull-down?

Componentes básicos

Antes de los sensores, estos son los "ladrillos" que aparecen en casi todos los proyectos.

LED (diodo emisor de luz)

Tiene polaridad: la **patita larga** es el **ánodo (+)**, la **corta** el **cátodo (-)**. Siempre con su resistencia en serie. Si lo conectas al revés, no prende.

Botón / interruptor

Conecta o corta el paso de corriente. Problema clásico: el **rebote** (un toque registra varios) → se corrige con un pequeño retardo o por software. Necesita pull-up/down.

Potenciómetro

Una resistencia variable (la perilla del volumen). Da un voltaje de 0 a 5V según la posición → ideal para practicar `analogRead()`.

Transistor y relé (para cargas grandes)

El Arduino no puede alimentar un motor o una ampolla directa. El **transistor** es un "interruptor electrónico": una señal chica del Arduino controla una corriente grande. El **relé** es un interruptor mecánico que permite manejar incluso 220V (con cuidado). Regla: **señal por el Arduino, potencia por una fuente aparte**.

Buzzer (zumbador)

- **Activo:** suena con solo darle corriente (un tono fijo).
- **Pasivo:** necesita una señal que oscile (`tone()`) y puede hacer melodías.

GEOGREEN

Nuestro buzzer usa `tone()` con un patrón "pip-pip" cada 5 minutos cuando el contenedor está lleno, en vez de un tono continuo (molestaba hasta a los perros 🐕). Eso vive en el código del firmware.

Ejercicios

1. ¿Cómo identificas el + de un LED nuevo (sin medir)?
2. ¿Por qué no conectar un motor directo a un pin del Arduino?
3. ¿Qué diferencia hay entre un buzzer activo y uno pasivo?

Sensores: el catálogo

Un sensor traduce algo del mundo físico (distancia, luz, temperatura...) a una señal eléctrica que el Arduino entiende. Lo importante de cada uno: **qué mide, qué tipo de salida da y cómo se conecta**.

Por su salida, los sensores son de tres tipos:

- **ANALÓGICA** entregan un voltaje variable → se leen con `analogRead()` (LDR, micrófono, MQ...).
- **DIGITAL** entregan 0/1 (¿hay movimiento? sí/no) → `digitalRead()` (PIR, sensor de línea...).
- **BUS** hablan por un protocolo (I2C/SPI/1-wire) y entregan datos ya procesados (DHT, MPU6050, BME280, RFID).

Distancia y proximidad

- **HC-SR04 (ultrasónico)**: manda un "ping" de sonido y mide cuánto tarda el eco → calcula distancia (2 cm a 4 m). Pines: VCC, GND, TRIG (dispara), ECHO (recibe). *El de GeoGreen*.
- **IR de línea (TCRT5000)**: detecta blanco/negro → sigue líneas (robots).
- **PIR (HC-SR501)**: detecta movimiento de cuerpos calientes (personas). Salida digital.

Ambientales

- **DHT11 / DHT22**: temperatura + humedad del aire (el DHT22 es más preciso). Salida por bus propio.
- **DS18B20**: temperatura (incluso sumergible), por 1-wire.
- **BME280**: temperatura, humedad y **presión** (altímetro). Por I2C.
- **LM35**: temperatura como voltaje analógico simple.

Luz y sonido

- **LDR (fotorresistencia)**: su resistencia cambia con la luz → con un divisor, da un valor analógico.
- **Micrófono (LM393)**: detecta nivel de sonido (aplausos, ruido).

Gas, humo y fuego

- **Serie MQ (MQ-2, MQ-135...)**: humo, gas, calidad de aire. Salida analógica.
- **Sensor de flama**: detecta luz infrarroja de una llama.

Agua

- **Humedad de suelo**: riego automático (¿la planta tiene sed?).
- **Sensor de lluvia / de nivel**: detecta agua o cuánta hay.

Movimiento, orientación y magnetismo

- **MPU6050**: acelerómetro + giroscopio (inclinación, vibración, drones). Por I2C.
- **Sensor de inclinación (tilt)**: ¿está derecho o volcado? Digital.
- **Sensor Hall**: detecta campos magnéticos (cuentavueeltas, finales de carrera).

Identificación

- **RFID RC522**: lee tarjetas/llaveros → control de acceso. Por SPI.

Tabla resumen

Sensor	Mide	Salida	Pines clave	Librería típica
HC-SR04	Distancia	Digital (pulso)	VCC·GND·TRIG·ECHO	NewPing
PIR HC-SR501	Movimiento	Digital	VCC·GND·OUT	—
DHT11/22	Temp + humedad	Bus 1-hilo	VCC·GND·DATA	DHT
BME280	Temp/hum/presión	I2C	SDA·SCL	Adafruit_BME280
LDR	Luz	Analógica	A0 (+ divisor)	—
MQ-2/135	Gas/humo	Analógica	A0 / DO	—
Humedad suelo	Agua en tierra	Analógica	A0	—
MPU6050	Aceleración/giro	I2C	SDA·SCL	MPU6050
RFID RC522	Tarjetas	SPI	MOSI·MISO·SCK·SS·RST	MFRC522

Ejercicios

1. Clasifica por salida (analógica/digital/bus): LDR, PIR, BME280.
2. ¿Qué sensor usarías para un riego automático? ¿Y para una alarma de movimiento?
3. El HC-SR04: ¿qué hace el pin TRIG y qué hace ECHO?

CAPÍTULO 9

Actuadores

Si el sensor es el "sentido", el actuador es el "músculo": convierte las decisiones del Arduino en acciones físicas.

Actuador	Hace	Cómo se controla	Ojo con...
LED / semáforo	Luz	digitalWrite / PWM	siempre con resistencia
Buzzer	Sonido	digitalWrite / tone()	activo vs pasivo
Motor DC	Girar (rueda)	driver L298N / L293D	fuentes aparte
Servo motor	Girar a un ángulo (0–180°)	librería Servo (PWM)	consumo en arranque
Motor paso a paso	Girar con precisión	driver (ULN2003/A4988)	cableado de bobinas
Relé	Encender cargas grandes/220V	digitalWrite + transistor	¡alto voltaje!
LCD 16×2 (I2C)	Mostrar texto	librería LiquidCrystal_I2C	2 cables (SDA/SCL)
OLED SSD1306	Mostrar gráficos	I2C / SPI	librería + memoria

GEOGREEN

La versión Arduino usa **3 LEDs (semáforo)** + **buzzer**. La versión ESP32 agrega una **pantalla OLED** para mostrar el % localmente.

Ejercicios

- ¿Qué actuador usarías para mover una compuerta a un ángulo exacto?
- ¿Por qué un relé necesita una fuente de poder aparte?
- ¿Cuántos cables de datos necesita una pantalla LCD por I2C?

Protocolos de comunicación

Muchos sensores no entregan un simple voltaje: "conversan" con el Arduino por un protocolo. Conocer cuál usan te dice **en qué pines van** y cuántos caben.

Protocolo	Cables	Pines en UNO	Velocidad	Bueno para
GPIO (digital/analógico)	1	cualquiera	—	LED, botón, sensores simples
UART / Serial	2 (TX/RX)	D0/D1 (+ USB)	media	hablar con el PC, GPS, Bluetooth
I2C	2 (SDA/SCL)	A4/A5	media	muchos sensores en 2 cables
SPI	4 (MOSI/MISO/SCK/SS)	D11/D12/D13/D10	alta	pantallas, RFID, tarjetas SD

LA VENTAJA DE I2C

Con **solo 2 cables** puedes conectar **varios** sensores a la vez (cada uno tiene una "dirección" única). Por eso es el favorito cuando el proyecto crece. **Serial** es el que usas siempre para depurar: `Serial.print()` manda mensajes a tu PC.

Ejercicios

1. Quieres 4 sensores ambientales sin quedarte sin pines. ¿Qué protocolo eliges?
2. ¿Qué protocolo usas, sin darte cuenta, cada vez que ves mensajes en el Monitor Serie?
3. ¿Cuántos cables necesita SPI y para qué tipo de dispositivos conviene?

Programación de Arduino

El código de Arduino siempre tiene la misma esqueleto: una función que corre **una vez** y otra que corre **para siempre**.

```
// Estructura mínima de TODO sketch de Arduino
void setup() { // corre 1 vez al encender
  pinMode(13, OUTPUT); // configura el pin 13 como salida
}

void loop() { // se repite infinitamente
  digitalWrite(13, HIGH); // prende
  delay(500); // espera 0,5 s
  digitalWrite(13, LOW); // apaga
  delay(500);
}
```

Las funciones que más usarás

Función	Qué hace
<code>pinMode(pin, MOD0)</code>	define un pin como INPUT / OUTPUT / INPUT_PULLUP
<code>digitalWrite(pin, HIGH/LOW)</code>	prende/apaga un pin digital
<code>digitalRead(pin)</code>	lee 0/1 de un pin
<code>analogRead(A0)</code>	lee 0–1023 de un pin analógico
<code>analogWrite(pin, 0–255)</code>	salida PWM (atenuar, velocidad)
<code>Serial.begin/print()</code>	habla con el PC (depurar)
<code>delay(ms) / millis()</code>	esperar / contar tiempo sin bloquear

DELAY() VS MILLIS()

`delay()` **congela** todo el programa mientras espera. Para que el Arduino haga varias cosas "a la vez" (medir y sonar y parpadear), se usa `millis()` (un cronómetro que no detiene nada). GeoGreen usa `millis()` para el patrón del buzzer.

Ejemplo: leer el HC-SR04

```

const int TRIG = 9, ECHO = 10;

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
}

void loop() {
  // 1) disparar el pulso ultrasónico
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  // 2) medir cuánto tarda el eco y pasarlo a cm
  long us = pulseIn(ECHO, HIGH);
  float cm = us / 58.0;
  Serial.print("Distancia: "); Serial.print(cm); Serial.println(" cm");
  delay(300);
}

```

Ejemplo: el semáforo por % (idea de GeoGreen)

```

// distancia -> % de llenado -> semáforo
int pct = map(cm, 100, 10, 0, 100); // 100cm=vacio, 10cm=lleno
pct = constrain(pct, 0, 100);

digitalWrite(4, pct < 40); // verde: bajo
digitalWrite(3, pct >= 40 && pct < 80); // amarillo: medio
digitalWrite(2, pct >= 80); // rojo: lleno
if (pct >= 80) tone(8, 2000); else noTone(8); // buzzer

```

Ejercicios

1. Modifica el Blink para que parpadee cada 2 segundos.
2. ¿Qué hace `map()` en el ejemplo del semáforo?
3. Explica por qué `millis()` es mejor que `delay()` cuando el Arduino debe vigilar y avisar a la vez.

IoT y ESP32 (introducción)

"IoT" (Internet de las Cosas) = objetos cotidianos conectados a la red que **envían datos** y se pueden controlar a distancia. Aquí solo lo presentamos; el foco de esta guía es Arduino clásico.

¿Qué cambia con el ESP32?

El ESP32 es un microcontrolador con **WiFi y Bluetooth integrados** y más potencia, a un precio similar. Programa casi igual que un Arduino (mismo IDE), pero trabaja a **3,3 voltios**. Permite que GeoGreen **suba el % de llenado a un dashboard** en internet.

EL DETALLE QUE QUEMA PLACAS: 5V VS 3,3V

El HC-SR04 entrega su señal **Echo en 5V**, pero un pin del ESP32 aguanta máximo **3,3V**. Conectarlo directo puede **dañar la placa**. La solución es un **divisor de voltaje** (dos resistencias, Cap. 6) que baja esos 5V a ~3,3V. En la UNO no hace falta porque ya trabaja a 5V.

Cómo viajan los datos

- **HTTP**: la placa "visita" una web/API y deja el dato (simple).
- **MQTT**: protocolo liviano para IoT: la placa "publica" lecturas y un servidor las reparte (eficiente para muchos dispositivos).

	Arduino UNO/Nano	ESP32
Voltaje	5V	3,3V
WiFi/Bluetooth	No	Sí
Potencia	Básica	Alta (doble núcleo)
Ideal para	aprender, control local	proyectos conectados (IoT)

GEOGREEN: LAS DOS PISTAS

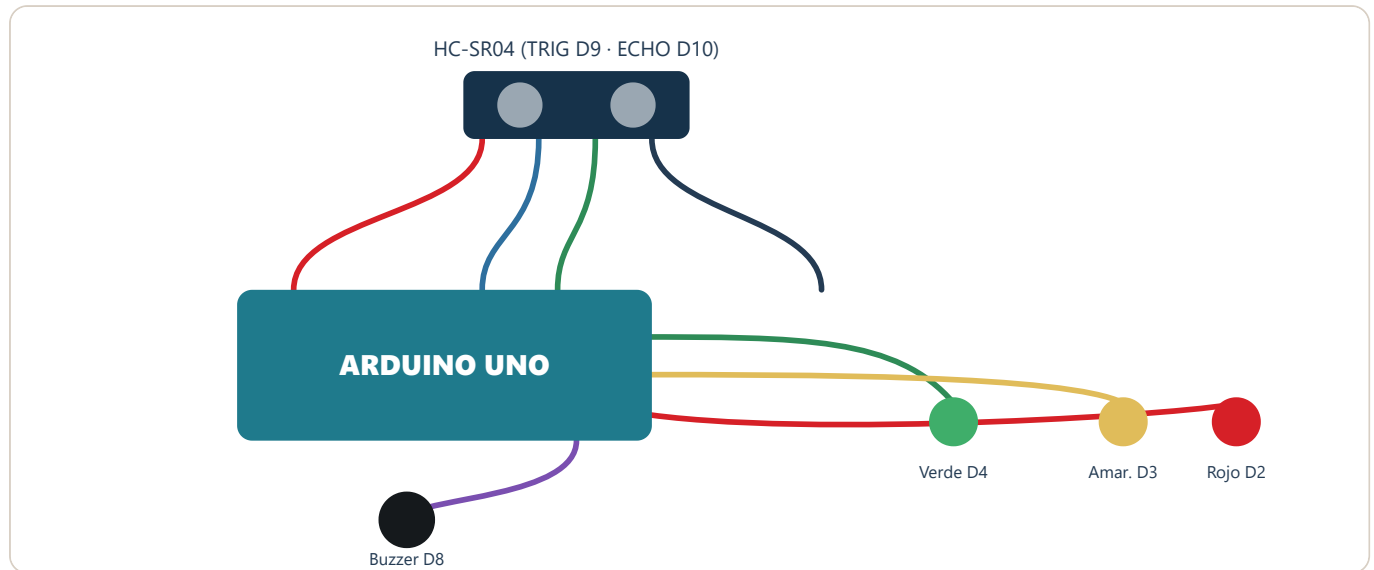
Pista Arduino (Don Elías): demo local con semáforo + buzzer, sin red. **Pista ESP32** (Diego): agrega OLED + envío del % a un dashboard, y a futuro georreferenciar puntos de reciclaje en un mapa — el "Geo" de GeoGreen.

Ejercicios

1. ¿Por qué hay que poner un divisor de voltaje al Echo del HC-SR04 en un ESP32?
2. Da un ejemplo de proyecto que necesite ESP32 y uno al que le baste una UNO.
3. ¿Qué ventaja tiene MQTT sobre HTTP cuando hay muchos dispositivos?

Caso práctico: el circuito GeoGreen

Juntemos todo. Este es el circuito completo del prototipo, con los pines reales del firmware.



Sensor → Arduino → semáforo + buzzer. Cada LED lleva su resistencia de $220\ \Omega$ (no dibujadas para simplificar).

La lógica completa:

1. El **HC-SR04** mide la distancia del tope del contenedor a la basura.
2. El **Arduino** convierte distancia → **% de llenado** (`map()`).
3. El **semáforo** muestra el estado: verde ($<40\%$), amarillo ($40\text{--}80\%$), rojo ($\geq 80\%$).
4. El **buzzer** avisa cuando está lleno (patrón pip-pip, sin molestar de más).
5. (Pista ESP32) Se envía el % a un **dashboard** por WiFi.

PRUÉBALO SIN PLACA

Todo esto corre en wokwi.com (simulador): pegas el código y el diagrama, le das Play y arrastras el control de distancia del sensor para ver cambiar el semáforo. Es lo que mostramos en el taller.

Seguridad y buenas prácticas

- **GND común:** todo lo que se conecte debe compartir la tierra del Arduino. Es el error #1.
- **Límite por pin (~20–40 mA):** LEDs y señales sí; motores/tiras LED no (usa fuente + transistor/relé).
- **Siempre resistencia al LED:** sin ella, se quema (y a veces el pin con él).
- **Respetar el voltaje:** no metas 5V a un pin de 3,3V (ESP32) sin divisor.
- **Conecta con la placa apagada:** arma el circuito, revisa, y recién alimenta.
- **No cruces + y –:** un cortocircuito puede dañar la placa o el USB del PC.
- **Cuidado con 220V:** los relés que manejan red eléctrica son para gente con experiencia.

CHECKLIST ANTES DE DAR ENERGÍA

¿GND común? ¿LEDs con resistencia? ¿+ y – en los rieles correctos? ¿Ningún componente cortocircuitado en la misma columna? ✓ Recién ahí, USB.

Ejercicios

1. Enumera 3 cosas que revisarías antes de alimentar un circuito nuevo.
2. ¿Por qué un motor no se conecta directo a un pin?
3. ¿Qué pasa si el sensor y el Arduino no comparten GND?

Glosario y recursos

Glosario

Término	Significado
Microcontrolador	Computador completo en un chip; el cerebro del Arduino.
Pin / GPIO	Conector de la placa para entrada/salida de señales.
Sketch	El programa que escribes y subes al Arduino.
Digital / Analógico	Dos estados (0/1) vs. valores variables.
PWM	Encender/apagar rápido para simular niveles intermedios.
Ley de Ohm	$V = I \times R$. Relaciona voltaje, corriente y resistencia.
Protoboard	Tablero para armar circuitos sin soldar.
Jumper	Cable puente para conectar componentes.
Pull-up / pull-down	Resistencia que fija un pin en un estado estable.
I2C / SPI / UART	Protocolos para que los sensores "conversen" con la placa.
Actuador	Componente que ejecuta una acción física (motor, LED, buzzer).
IoT	Objetos conectados a internet que envían/reciben datos.

Para seguir aprendiendo

- **arduino.cc** — documentación oficial, referencia del lenguaje y de placas.
- **wokwi.com** — simulador online: arma y corre circuitos sin placa física.
- **Random Nerd Tutorials** — guías gratuitas de sensores y módulos (Arduino y ESP32).
- **CircuitDigest / Electronics Hub** — comparativas de placas y proyectos.
- El repositorio **GeoGreen** — código, diagrama Wokwi y la viz 3D del proyecto.

CIERRE

Con esto tienes el mapa completo: qué es Arduino, cómo elegir placa, cómo funciona la electricidad y la protoboard, qué sensores existen y cómo se programan. Lo demás es **practicar** — y para eso, GeoGreen es el mejor pretexto. 🌱